

API Specification

Spring Boot Boilerplate
Enterprise Multi-Tenant Expense Management Platform

Version 1.0

Generated: August 2026

Base URL: <http://localhost:8080>

Authentication: JWT Bearer Token (stateless)

Table of Contents

1. **Overview** - System architecture and general information
2. **Authentication** - Login, MFA, token refresh, password management
3. **Expenses** - Create, update, approve, reject, and process expenses
4. **User Management** - CRUD operations, role assignment, MFA management
5. **Role Management** - Create, update, and manage roles and permissions
6. **Tenant Management** - Multi-tenant administration
7. **Department Management** - Organizational structure management
8. **Audit Log** - System audit trail and compliance logging
9. **System / Actuator** - Health checks, metrics, and monitoring
10. **Security Model** - Permissions, roles, and authorization framework
11. **Error Handling** - Standard error responses and codes

1. Overview

This document provides a comprehensive specification of all available API endpoints for the Spring Boot Boilerplate platform. The platform is a multi-tenant enterprise expense management system built with Java 21, Spring Boot 3.2.5, and secured with stateless JWT authentication.

Technology Stack

Component	Details
Runtime	Java 21, Spring Boot 3.2.5, Maven
Database	H2 (dev/test), PostgreSQL (production), Flyway migrations
Security	Spring Security, JWT (stateless), Method-level authorization
Caching	Redis
Monitoring	Spring Actuator, Prometheus metrics
Testing	JUnit 5, Mockito, AssertJ

Base URL and Response Format

All API endpoints are served from the root path (no context path prefix). All responses are wrapped in a standard envelope:

```
{"success": boolean, "message": string, "data": T, "timestamp": "ISO-8601"}
```

Endpoint Summary

Metric	Count / Detail
Total Endpoints	42 custom + 4 actuator
Public Endpoints	5 (login, refresh, forgot-password, reset-password, mfa/verify)
Authenticated-only	3 (logout, /me, change-password)
Permission-gated	34 (via @PreAuthorize on service methods)

2. Authentication

This section documents all authentication endpoints. There are 8 endpoint(s) in this group.

POST /api/auth/login

Summary: Authenticate a user and issue JWT tokens

Authorization: None (Public)

Validates credentials and returns an access token. If multi-factor authentication is enabled for the user, returns an MFA session token instead. The refresh token is set as an HttpOnly Secure cookie.

Request Body

Field	Type	Description	Validation
usernameOrEmail	string	Required. The username or email address.	NotBlank
password	string	Required. The user password.	NotBlank, Size(1-100)
rememberMe	boolean	Optional. Extends refresh token lifetime.	None

Response: TokenResponse or MfaLoginResponse

Response Fields

Field	Type	Description
accessToken	string	The JWT access token.
tokenType	string	Always 'Bearer'.
expiresIn	long	Token lifetime in seconds.
username	string	Authenticated username.
roles	string[]	Granted role names.

Status Codes

Status Code
200 OK
401 Unauthorized
429 Too Many Requests (account locked)

POST /api/auth/mfa/verify

Summary: Verify multi-factor authentication code

Authorization: None (Public)

Completes the MFA challenge using the session token and a 6-digit code. Returns JWT tokens on success.

Request Body

Field	Type	Description	Validation
mfaSessionToken	string	Required. Session token from login.	NotBlank
code	string	Required. 6-digit verification code.	NotBlank, Size(6,6)

Response: TokenResponse

Response Fields

Field	Type	Description
accessToken	string	The JWT access token.
tokenType	string	Always 'Bearer'.
expiresIn	long	Token lifetime in seconds.
username	string	Authenticated username.
roles	string[]	Granted role names.

Status Codes

Status Code
200 OK
401 Unauthorized

POST /api/auth/refresh

Summary: Refresh an expired access token

Authorization: None (Public - uses HttpOnly cookie)

Rotates the refresh token stored in the HttpOnly cookie and issues a new access token. The previous refresh token is invalidated.

Response: TokenResponse

Response Fields

Field	Type	Description
accessToken	string	New JWT access token.
tokenType	string	Always 'Bearer'.
expiresIn	long	Token lifetime in seconds.
username	string	Authenticated username.
roles	string[]	Granted role names.

Status Codes

Status Code
200 OK
401 Unauthorized (invalid/expired refresh token)

POST /api/auth/logout

Summary: Invalidate the current session

Authorization: Authenticated (JWT required)

Blacklists the current access token, clears the refresh token cookie, and records a security audit event.

Response: Void

Status Codes

Status Code
200 OK
401 Unauthorized

GET /api/auth/me

Summary: Retrieve the current user profile

Authorization: Authenticated (JWT required)

Returns the profile details of the currently authenticated user including roles and MFA status.

Response: UserProfileResponse

Response Fields

Field	Type	Description
username	string	The username.
email	string	The email address.
firstName	string	First name.
lastName	string	Last name.
roles	string[]	Assigned role names.
enabled	boolean	Account enabled status.
mfaEnabled	boolean	Whether MFA is active.
mfaMethod	string	MFA method (NONE, TOTP, EMAIL).

Status Codes

Status Code
200 OK
401 Unauthorized

POST /api/auth/change-password

Summary: Change the current user password

Authorization: Authenticated (JWT required)

Validates the current password, sets the new password, and clears all refresh tokens for the user.

Request Body

Field	Type	Description	Validation
currentPassword	string	Required. The existing password.	NotBlank
newPassword	string	Required. The new password.	NotBlank, @Password
confirmPassword	string	Required. Must match newPassword.	NotBlank

Response: Void

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized

POST /api/auth/forgot-password

Summary: Request a password reset link

Authorization: None (Public)

Sends a password reset email if the provided email address exists in the system. Returns success regardless to prevent email enumeration.

Request Body

Field	Type	Description	Validation
email	string	Required. The account email address.	NotBlank, Email

Response: Void

Status Codes

Status Code
200 OK

POST /api/auth/reset-password

Summary: Reset password using a token

Authorization: None (Public)

Resets the user password using a single-use token received via email. The token has a configurable time-to-live.

Request Body

Field	Type	Description	Validation
token	string	Required. The password reset token.	NotBlank
newPassword	string	Required. The new password.	NotBlank, @Password
confirmPassword	string	Required. Must match newPassword.	NotBlank

Response: Void

Status Codes

Status Code
200 OK
400 Bad Request (invalid/expired token)

3. Expenses

This section documents all expenses endpoints. There are 8 endpoint(s) in this group.

GET /api/expenses

Summary: List expenses with optional filters

Authorization: Authenticated - requires EXPENSE_READ

Returns a paginated list of expenses. Tenant-scoped: regular users see only their tenant expenses; super admin sees all. Supports filtering by status, tenant, and department.

Query Parameters

Parameter	Type	Description
status	string	Optional. Filter by status: PENDING, APPROVED, REJECTED, CANCELLED, PROCESSED.
tenantId	long	Optional. Filter by tenant (super admin only).
departmentId	long	Optional. Filter by department.
page	int	Optional. Page number (0-based). Default: 0.
size	int	Optional. Page size. Default: 20.
sort	string	Optional. Sort field and direction.

Response: Page of ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
title	string	Expense title.
description	string	Expense description.
amount	decimal	Expense amount.
category	string	Expense category.
status	string	Current status.
submissionDate	datetime	When the expense was submitted.
ownerUsername	string	Username of the expense owner.
departmentName	string	Department name.
tenantName	string	Tenant name.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden

GET /api/expenses/{id}

Summary: Retrieve a single expense by ID

Authorization: Authenticated - requires EXPENSE_READ

Returns the full details of a specific expense. Includes additional ownership and tenant access checks.

Path Parameters

Parameter	Type	Description
id	long	Required. The expense ID.

Response: ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
title	string	Expense title.
description	string	Expense description.
amount	decimal	Expense amount.
category	string	Expense category.
status	string	Current status.
submissionDate	datetime	When the expense was submitted.
decisionDate	datetime	When approved or rejected.
processedDate	datetime	When processed by finance.
ownerUsername	string	Username of the owner.
approvedByUsername	string	Username of the approver.
rejectedByUsername	string	Username of the rejector.
processedByUsername	string	Username of the processor.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

POST /api/expenses

Summary: Submit a new expense

Authorization: Authenticated - requires EXPENSE_CREATE

Creates a new expense in PENDING status. The authenticated user becomes the owner. Tenant membership is required.

Request Body

Field	Type	Description	Validation
title	string	Required. Expense title.	NotBlank, Size(max=200)
description	string	Optional. Expense description.	Size(max=1000)
amount	decimal	Required. Positive amount.	NotNull, Positive, Digits(12,4)
category	string	Required. Expense category.	NotBlank, Size(max=50)
departmentId	long	Optional. Associated department ID.	None

Response: ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
title	string	Expense title.
amount	decimal	Expense amount.
status	string	Always PENDING on creation.
submissionDate	datetime	Creation timestamp.

Status Codes

Status Code
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden

PUT /api/expenses/{id}

Summary: Update an existing expense

Authorization: Authenticated - requires EXPENSE_UPDATE

Updates the title, description, amount, or category of an expense. Only PENDING expenses can be modified. Ownership check is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The expense ID.

Request Body

Field	Type	Description	Validation
title	string	Required. Expense title.	NotBlank, Size(max=200)
description	string	Optional. Expense description.	Size(max=1000)
amount	decimal	Required. Positive amount.	NotNull, Positive, Digits(12,4)
category	string	Required. Expense category.	NotBlank, Size(max=50)

Response: ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
title	string	Updated title.
amount	decimal	Updated amount.
status	string	Must be PENDING.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
409 Conflict

POST /api/expenses/{id}/cancel

Summary: Cancel a pending expense

Authorization: Authenticated - requires EXPENSE_UPDATE

Sets the expense status to CANCELLED. Only the owner of a PENDING expense may cancel it.

Path Parameters

Parameter	Type	Description
id	long	Required. The expense ID.

Response: ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
status	string	Updated to CANCELLED.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
409 Conflict

POST /api/expenses/{id}/approve

Summary: Approve a pending expense

Authorization: Authenticated - requires EXPENSE_APPROVE

Approves a PENDING expense. The approver must have appropriate authority and cannot approve their own expense.

Path Parameters

Parameter	Type	Description
id	long	Required. The expense ID.

Response: ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
status	string	Updated to APPROVED.
approvedByUsername	string	Username of the approver.
decisionDate	datetime	Approval timestamp.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
409 Conflict

POST /api/expenses/{id}/reject

Summary: Reject a pending expense

Authorization: Authenticated - requires EXPENSE_REJECT

Rejects a PENDING expense. The rejector must have appropriate authority.

Path Parameters

Parameter	Type	Description
id	long	Required. The expense ID.

Response: ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
status	string	Updated to REJECTED.
rejectedByUsername	string	Username of the rejector.
decisionDate	datetime	Rejection timestamp.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
409 Conflict

POST /api/expenses/{id}/process

Summary: Process an approved expense

Authorization: Authenticated - requires EXPENSE_PROCESS

Marks an APPROVED expense as PROCESSED by the finance team. Only APPROVED expenses can be processed.

Path Parameters

Parameter	Type	Description
id	long	Required. The expense ID.

Response: ExpenseResponse

Response Fields

Field	Type	Description
id	long	Unique expense identifier.
status	string	Updated to PROCESSED.
processedByUsername	string	Username of the processor.
processedDate	datetime	Processing timestamp.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
409 Conflict

4. User Management

This section documents all user management endpoints. There are 11 endpoint(s) in this group.

POST /api/management/users

Summary: Create a new user

Authorization: Authenticated - requires USER_CREATE

Creates a new user account with the specified role and department. Duplicate username or email values are rejected.

Request Body

Field	Type	Description	Validation
username	string	Required. Unique username.	NotBlank, Size(3-50)
email	string	Required. Unique email.	NotBlank, Email, Size(max=100)
password	string	Required. User password.	NotBlank, @Password
firstName	string	Optional. First name.	Size(max=50)
lastName	string	Optional. Last name.	Size(max=50)
roleName	string	Optional. Role name to assign.	Size(max=50)
tenantId	long	Optional. Tenant ID.	None
departmentId	long	Required. Department ID.	NotNull

Response: UserResponse

Response Fields

Field	Type	Description
id	long	Unique user identifier.
username	string	The username.
email	string	The email address.
firstName	string	First name.
lastName	string	Last name.
enabled	boolean	Account enabled status.
departmentName	string	Assigned department name.
roles	string[]	Assigned role names.
permissions	string[]	Effective permission names.
mfaEnabled	boolean	MFA status.
createdAt	datetime	Account creation timestamp.

Status Codes

Status Code
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden

GET /api/management/users

Summary: List users with pagination

Authorization: Authenticated - requires USER_READ

Returns a paginated list of users. Super admin sees all users; tenant admin sees only users within their tenant.

Query Parameters

Parameter	Type	Description
page	int	Optional. Page number (0-based). Default: 0.
size	int	Optional. Page size. Default: 20.
sort	string	Optional. Sort field and direction.

Response: Page of UserResponse

Response Fields

Field	Type	Description
id	long	Unique user identifier.
username	string	The username.
email	string	The email address.
enabled	boolean	Account enabled status.
departmentName	string	Assigned department name.
roles	string[]	Assigned role names.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden

GET /api/management/users/{id}

Summary: Retrieve a user by ID

Authorization: Authenticated - requires USER_READ

Returns the full details of a specific user. Tenant-scoped access is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Response: UserResponse

Response Fields

Field	Type	Description
id	long	Unique user identifier.
username	string	The username.
email	string	The email address.
firstName	string	First name.
lastName	string	Last name.
enabled	boolean	Account enabled status.
accountNonLocked	boolean	Account lock status.
departmentName	string	Assigned department name.
roles	string[]	Assigned role names.
permissions	string[]	Effective permission names.
mfaEnabled	boolean	MFA status.
mfaMethod	string	MFA method (NONE, TOTP, EMAIL).
createdAt	datetime	Account creation timestamp.
updatedAt	datetime	Last update timestamp.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

PUT /api/management/users/{id}

Summary: Update a user profile

Authorization: Authenticated - requires USER_WRITE

Updates the first name, last name, or department of a user. Privilege hierarchy is enforced: a user cannot modify someone with equal or higher privileges.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Request Body

Field	Type	Description	Validation
firstName	string	Optional. First name.	Size(max=50)
lastName	string	Optional. Last name.	Size(max=50)
departmentId	long	Optional. New department ID.	None

Response: UserResponse

Response Fields

Field	Type	Description
id	long	Unique user identifier.
firstName	string	Updated first name.
lastName	string	Updated last name.
departmentName	string	Updated department name.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found

DELETE /api/management/users/{id}

Summary: Delete a user

Authorization: Authenticated - requires USER_DELETE

Permanently removes a user account. Cannot delete yourself, cannot delete the last admin in a tenant, and privilege hierarchy is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Response: Void

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict

POST /api/management/users/{id}/enable

Summary: Enable or disable a user account

Authorization: Authenticated - requires USER_ENABLE

Toggles the enabled status of a user account. Cannot change your own status, and cannot disable the last admin in a tenant.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Request Body

Field	Type	Description	Validation
enabled	boolean	Required. True to enable, false to disable.	NotNull

Response: UserResponse

Response Fields

Field	Type	Description
id	long	Unique user identifier.
enabled	boolean	Updated enabled status.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict

POST /api/management/users/{id}/roles

Summary: Assign a role to a user

Authorization: Authenticated - requires USER_ASSIGN_ROLE

Assigns the specified role to a user. Role permission validation and privilege hierarchy checks are enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Request Body

Field	Type	Description	Validation
roleName	string	Required. The role name to assign.	NotBlank

Response: UserResponse

Response Fields

Field	Type	Description
id	long	Unique user identifier.
roles	string[]	Updated role names.
permissions	string[]	Updated effective permissions.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found

DELETE /api/management/users/{id}/roles

Summary: Remove a role from a user

Authorization: Authenticated - requires USER_ASSIGN_ROLE

Removes the specified role from a user. Cannot remove the last admin role, and permission hierarchy is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Request Body

Field	Type	Description	Validation
roleName	string	Required. The role name to remove.	NotBlank

Response: UserResponse

Response Fields

Field	Type	Description
id	long	Unique user identifier.
roles	string[]	Updated role names.
permissions	string[]	Updated effective permissions.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict

POST /api/management/users/{id}/mfa/enable

Summary: Enable multi-factor authentication for a user

Authorization: Authenticated - requires USER_WRITE

Enables MFA for the specified user using the chosen method (TOTP or EMAIL). Returns setup details including QR URI for authenticator apps.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Request Body

Field	Type	Description	Validation
method	string	Required. MFA method: TOTP or EMAIL.	NotNull

Response: MfaSetupResponse

Response Fields

Field	Type	Description
qrUri	string	QR code URI for authenticator apps.
secret	string	MFA secret key.
method	string	Configured MFA method.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

POST /api/management/users/{id}/mfa/disable

Summary: Disable multi-factor authentication

Authorization: Authenticated - requires USER_WRITE

Disables MFA for the specified user. Privilege hierarchy is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Response: Void

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

POST /api/management/users/{id}/mfa/reset

Summary: Reset MFA configuration for a user

Authorization: Authenticated - requires USER_WRITE

Resets the existing MFA configuration and sets up a new one with the specified method. Returns new setup details.

Path Parameters

Parameter	Type	Description
id	long	Required. The user ID.

Request Body

Field	Type	Description	Validation
method	string	Required. New MFA method: TOTP or EMAIL.	NotNull

Response: MfaSetupResponse

Response Fields

Field	Type	Description
qrUri	string	New QR code URI.
secret	string	New MFA secret key.
method	string	Configured MFA method.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

5. Role Management

This section documents all role management endpoints. There are 7 endpoint(s) in this group.

GET /api/management/roles

Summary: List all roles

Authorization: Authenticated - requires ROLE_READ

Returns a paginated list of all roles in the system including their assigned permissions.

Query Parameters

Parameter	Type	Description
page	int	Optional. Page number (0-based). Default: 0.
size	int	Optional. Page size. Default: 20.
sort	string	Optional. Sort field and direction.

Response: Page of RoleResponse

Response Fields

Field	Type	Description
id	long	Unique role identifier.
name	string	Role name (e.g., EMPLOYEE).
title	string	Human-readable role title.
description	string	Role description.
permissions	string[]	Assigned permission names.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden

GET /api/management/roles/{id}

Summary: Retrieve a role by ID

Authorization: Authenticated - requires ROLE_READ

Returns the full details of a specific role including all assigned permissions.

Path Parameters

Parameter	Type	Description
id	long	Required. The role ID.

Response: RoleResponse

Response Fields

Field	Type	Description
id	long	Unique role identifier.
name	string	Role name.
title	string	Human-readable title.
description	string	Role description.
permissions	string[]	Assigned permission names.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

POST /api/management/roles

Summary: Create a new role

Authorization: Authenticated - requires PLATFORM_ADMIN role

Creates a new custom role. Only platform administrators can create roles. Built-in roles cannot be duplicated.

Request Body

Field	Type	Description	Validation
name	string	Required. Unique role name.	NotBlank, Size(max=50)
title	string	Optional. Human-readable title.	Size(max=100)
description	string	Optional. Role description.	Size(max=255)

Response: RoleResponse

Response Fields

Field	Type	Description
id	long	Unique role identifier.
name	string	Created role name.
title	string	Human-readable title.
description	string	Role description.

Status Codes

Status Code
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden

PUT /api/management/roles/{id}

Summary: Update a role

Authorization: Authenticated - requires PLATFORM_ADMIN role

Updates the name, title, or description of a role. Built-in roles cannot be modified.

Path Parameters

Parameter	Type	Description
id	long	Required. The role ID.

Request Body

Field	Type	Description	Validation
name	string	Required. Updated role name.	NotBlank, Size(max=50)
title	string	Optional. Updated title.	Size(max=100)
description	string	Optional. Updated description.	Size(max=255)

Response: RoleResponse

Response Fields

Field	Type	Description
id	long	Unique role identifier.
name	string	Updated role name.
title	string	Updated title.
description	string	Updated description.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
409 Conflict

DELETE /api/management/roles/{id}

Summary: Delete a role

Authorization: Authenticated - requires PLATFORM_ADMIN role

Permanently removes a role. Built-in roles and roles currently assigned to users cannot be deleted.

Path Parameters

Parameter	Type	Description
id	long	Required. The role ID.

Response: Void

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict

POST /api/management/roles/{id}/permissions

Summary: Add a permission to a role

Authorization: Authenticated - requires PLATFORM_ADMIN role

Adds a permission to the specified role. Built-in role permissions cannot be modified.

Path Parameters

Parameter	Type	Description
id	long	Required. The role ID.

Request Body

Field	Type	Description	Validation
permission	string	Required. Permission name from UserPermission enum.	NotNull

Response: RoleResponse

Response Fields

Field	Type	Description
id	long	Unique role identifier.
permissions	string[]	Updated permission set.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
409 Conflict

DELETE /api/management/roles/{id}/permissions

Summary: Remove a permission from a role

Authorization: Authenticated - requires PLATFORM_ADMIN role

Removes a permission from the specified role. Built-in role permissions cannot be modified.

Path Parameters

Parameter	Type	Description
id	long	Required. The role ID.

Request Body

Field	Type	Description	Validation
permission	string	Required. Permission name to remove.	NotNull

Response: RoleResponse

Response Fields

Field	Type	Description
id	long	Unique role identifier.
permissions	string[]	Updated permission set.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
409 Conflict

6. Tenant Management

This section documents all tenant management endpoints. There are 5 endpoint(s) in this group.

GET /api/management/tenants

Summary: List all tenants

Authorization: Authenticated - requires TENANT_READ

Returns a paginated list of tenants. Only platform administrators can list tenants. Supports optional name filtering.

Query Parameters

Parameter	Type	Description
name	string	Optional. Filter by tenant name.
page	int	Optional. Page number (0-based). Default: 0.
size	int	Optional. Page size. Default: 20.
sort	string	Optional. Sort field and direction.

Response: Page of TenantResponse

Response Fields

Field	Type	Description
id	long	Unique tenant identifier.
name	string	Tenant name.
status	string	Tenant status (ACTIVE, INACTIVE, SUSPENDED).
createdAt	datetime	Creation timestamp.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden

GET /api/management/tenants/{id}

Summary: Retrieve a tenant by ID

Authorization: Authenticated - requires TENANT_READ

Returns the full details of a specific tenant. Tenant access is checked via AuthorizationService.

Path Parameters

Parameter	Type	Description
id	long	Required. The tenant ID.

Response: TenantResponse

Response Fields

Field	Type	Description
id	long	Unique tenant identifier.
name	string	Tenant name.
status	string	Tenant status.
createdAt	datetime	Creation timestamp.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

POST /api/management/tenants

Summary: Create a new tenant

Authorization: Authenticated - requires TENANT_CREATE

Creates a new tenant with the specified name and status. Duplicate tenant names are rejected.

Request Body

Field	Type	Description	Validation
name	string	Required. Unique tenant name.	NotBlank, Size(max=100)
status	string	Required. Initial status: ACTIVE, INACTIVE, or SUSPENDED.	NotNull

Response: TenantResponse

Response Fields

Field	Type	Description
id	long	Unique tenant identifier.
name	string	Created tenant name.
status	string	Initial status.
createdAt	datetime	Creation timestamp.

Status Codes

Status Code
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden

PUT /api/management/tenants/{id}

Summary: Update a tenant

Authorization: Authenticated - requires TENANT_UPDATE

Updates the name or status of a tenant. Tenant management access is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The tenant ID.

Request Body

Field	Type	Description	Validation
name	string	Required. Updated tenant name.	NotBlank, Size(max=100)
status	string	Required. Updated status.	NotNull

Response: TenantResponse

Response Fields

Field	Type	Description
id	long	Unique tenant identifier.
name	string	Updated tenant name.
status	string	Updated status.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
409 Conflict

DELETE /api/management/tenants/{id}

Summary: Delete a tenant

Authorization: Authenticated - requires TENANT_DELETE

Permanently removes a tenant. Tenant management access is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The tenant ID.

Response: Void

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

7. Department Management

This section documents all department management endpoints. There are 5 endpoint(s) in this group.

GET /api/management/departments

Summary: List departments with pagination

Authorization: Authenticated - requires DEPARTMENT_READ

Returns a paginated list of departments. Super admin sees all; tenant admin sees only their tenant departments.

Query Parameters

Parameter	Type	Description
page	int	Optional. Page number (0-based). Default: 0.
size	int	Optional. Page size. Default: 20.
sort	string	Optional. Sort field and direction.

Response: Page of DepartmentResponse

Response Fields

Field	Type	Description
id	long	Unique department identifier.
name	string	Department name.
tenantId	long	Parent tenant ID.
tenantName	string	Parent tenant name.
managerIds	long[]	List of manager user IDs.
managerUsernames	string[]	List of manager usernames.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden

GET /api/management/departments/{id}

Summary: Retrieve a department by ID

Authorization: Authenticated - requires DEPARTMENT_READ

Returns the full details of a specific department including manager information. Tenant-scoped access is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The department ID.

Response: DepartmentResponse

Response Fields

Field	Type	Description
id	long	Unique department identifier.
name	string	Department name.
tenantId	long	Parent tenant ID.
tenantName	string	Parent tenant name.
managerIds	long[]	Manager user IDs.
managerUsernames	string[]	Manager usernames.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

POST /api/management/departments

Summary: Create a new department

Authorization: Authenticated - requires DEPARTMENT_CREATE

Creates a new department within the specified tenant. Duplicate department names within the same tenant are rejected.

Request Body

Field	Type	Description	Validation
name	string	Required. Department name.	NotBlank, Size(max=100)
tenantId	long	Required. Parent tenant ID.	NotNull
managerIds	long[]	Optional. List of manager user IDs.	None

Response: DepartmentResponse

Response Fields

Field	Type	Description
id	long	Unique department identifier.
name	string	Created department name.
tenantName	string	Parent tenant name.
managerUsernames	string[]	Manager usernames.

Status Codes

Status Code
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden

PUT /api/management/departments/{id}

Summary: Update a department

Authorization: Authenticated - requires DEPARTMENT_UPDATE

Updates the name or manager list of a department. Department management access is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The department ID.

Request Body

Field	Type	Description	Validation
name	string	Required. Updated department name.	NotBlank, Size(max=100)
managerIds	long[]	Optional. Updated manager user IDs.	None

Response: DepartmentResponse

Response Fields

Field	Type	Description
id	long	Unique department identifier.
name	string	Updated department name.
managerUsernames	string[]	Updated manager usernames.

Status Codes

Status Code
200 OK
400 Bad Request
401 Unauthorized
403 Forbidden
409 Conflict

DELETE /api/management/departments/{id}

Summary: Delete a department

Authorization: Authenticated - requires DEPARTMENT_DELETE

Permanently removes a department. Department management access is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The department ID.

Response: Void

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

8. Audit Log

This section documents all audit log endpoints. There are 2 endpoint(s) in this group.

GET /api/management/audit

Summary: List audit log entries

Authorization: Authenticated - requires AUDIT_LOG_READ

Returns a paginated list of audit log entries. Super admin sees all entries; tenant admin sees only their tenant entries.

Query Parameters

Parameter	Type	Description
page	int	Optional. Page number (0-based). Default: 0.
size	int	Optional. Page size. Default: 20.
sort	string	Optional. Sort field and direction.

Response: Page of AuditLogResponse

Response Fields

Field	Type	Description
id	long	Unique log entry identifier.
actorId	long	ID of the user who performed the action.
actorUsername	string	Username of the actor.
tenantId	long	Tenant where the action occurred.
action	string	Action performed (e.g., USER_CREATED, EXPENSE_APPROVED).
resourceType	string	Type of resource affected (USER, EXPENSE, TENANT).
resourceId	string	ID of the affected resource.
details	string	Additional details about the action.
timestamp	datetime	When the action occurred.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden

GET /api/management/audit/{id}

Summary: Retrieve a single audit log entry

Authorization: Authenticated - requires AUDIT_LOG_READ

Returns the full details of a specific audit log entry. Tenant-scoped access is enforced.

Path Parameters

Parameter	Type	Description
id	long	Required. The audit log entry ID.

Response: AuditLogResponse

Response Fields

Field	Type	Description
id	long	Unique log entry identifier.
actorId	long	ID of the actor.
actorUsername	string	Username of the actor.
tenantId	long	Tenant ID.
action	string	Action performed.
resourceType	string	Resource type.
resourceId	string	Resource ID.
details	string	Action details.
timestamp	datetime	Action timestamp.

Status Codes

Status Code
200 OK
401 Unauthorized
403 Forbidden
404 Not Found

9. System / Actuator

This section documents all system / actuator endpoints. There are 4 endpoint(s) in this group.

GET /actuator/health

Summary: Application health check

Authorization: None (Public)

Returns the health status of the application and its dependencies. Basic status is always available; detailed information requires authentication.

Response: HealthIndicator

Response Fields

Field	Type	Description
status	string	UP, DOWN, or OUT_OF_SERVICE.
components	object	Detailed health of each dependency (when authorized).

Status Codes

Status Code
200 OK

GET /actuator/info

Summary: Application information

Authorization: Authenticated

Returns general application information and metadata.

Response: Info

Response Fields

Field	Type	Description
build	object	Build information (version, artifact).
git	object	Git commit information.

Status Codes

Status Code
200 OK
401 Unauthorized

GET /actuator/metrics

Summary: Application metrics

Authorization: Authenticated

Returns a list of all available metric names for monitoring.

Response: Metrics

Response Fields

Field	Type	Description
names	string[]	Available metric names.

Status Codes

Status Code
200 OK
401 Unauthorized

GET /actuator/prometheus

Summary: Prometheus metrics endpoint

Authorization: Authenticated

Returns application metrics in Prometheus exposition format for scraping by a Prometheus server.

Response: text/plain (Prometheus format)

Status Codes

Status Code
200 OK
401 Unauthorized

10. Security Model

The platform implements a comprehensive role-based access control (RBAC) system with multi-tenancy. Authorization is enforced at the service layer using Spring Security method-level annotations.

Built-in Roles

Role Name	Description
PLATFORM_ADMIN	Full system access, bypasses tenant isolation
TENANT_ADMIN	Full access within assigned tenant
USER_MANAGER	Manage users within tenant
DEPARTMENT_MANAGER	Manage assigned departments
EMPLOYEE	Basic access - submit and view own expenses
AUDITOR	Read-only access to audit logs and reports
FINANCE	Process approved expenses

Available Permissions

Permission	Description
TENANT_READ	Read tenant information
TENANT_CREATE	Create new tenants
TENANT_UPDATE	Update tenant details
TENANT_DELETE	Remove tenants
USER_READ	View user accounts
USER_WRITE	Modify user profiles and MFA settings
USER_CREATE	Create new user accounts
USER_UPDATE	Update user information
USER_DELETE	Remove user accounts
USER_ENABLE	Enable or disable user accounts
USER_ASSIGN_ROLE	Assign or remove roles from users
ROLE_READ	View role definitions
ROLE_WRITE	Modify role details
ROLE_DELETE	Remove custom roles
ROLE_ASSIGN_PERMISSION	Add or remove permissions from roles
DEPARTMENT_READ	View department information
DEPARTMENT_CREATE	Create new departments
DEPARTMENT_UPDATE	Modify department details
DEPARTMENT_DELETE	Remove departments
EXPENSE_READ	View expenses
EXPENSE_READ_ALL	View all expenses across tenants
EXPENSE_CREATE	Submit new expenses
EXPENSE_UPDATE	Modify or cancel pending expenses
EXPENSE_DELETE	Delete expenses
EXPENSE_APPROVE	Approve pending expenses
EXPENSE_REJECT	Reject pending expenses
EXPENSE_PROCESS	Process approved expenses
MFA_MANAGE	Manage multi-factor authentication
REPORT_READ	View reports

Permission	Description
AUDIT_LOG_READ	View audit log entries

Audit Actions

The following actions are recorded in the audit log for compliance and traceability:

Action	Description
USER_CREATED	A new user account was created
USER_UPDATED	A user profile was modified
USER_DELETED	A user account was removed
USER_ENABLED	A user account was enabled
USER_DISABLED	A user account was disabled
USER_ROLE_ASSIGNED	A role was assigned to a user
USER_ROLE_REMOVED	A role was removed from a user
USER_MFA_ENABLED	MFA was enabled for a user
USER_MFA_DISABLED	MFA was disabled for a user
USER_MFA_RESET	MFA was reset for a user
EXPENSE_CREATED	A new expense was submitted
EXPENSE_UPDATED	An expense was modified
EXPENSE_CANCELLED	An expense was cancelled
EXPENSE_APPROVED	An expense was approved
EXPENSE_REJECTED	An expense was rejected
EXPENSE_PROCESSED	An expense was processed by finance
TENANT_CREATED	A new tenant was created
TENANT_UPDATED	A tenant was modified
TENANT_DELETED	A tenant was removed

11. Error Handling

All error responses follow the standard API envelope with `success=false`. The `GlobalExceptionHandler` catches standard Java exceptions and maps them to appropriate HTTP status codes. No custom exception classes are used.

Standard Error Codes

HTTP Status	Cause	Exception Type
400 Bad Request	Validation failed, duplicate constraint, or invalid input	<code>MethodArgumentNotValidException</code> , <code>DataIntegrityViolationException</code> , <code>IllegalArgumentException</code>
401 Unauthorized	Invalid or missing credentials	<code>BadCredentialsException</code> , <code>UsernameNotFoundException</code>
403 Forbidden	Insufficient permissions or tenant boundary violation	<code>AccessDeniedException</code>
409 Conflict	Invalid state transition (e.g., approving already-rejected expense)	<code>IllegalStateException</code>
429 Too Many Requests	Account locked due to too many failed login attempts	<code>LockedException</code>
500 Internal Server Error	Unexpected server error	Exception (catch-all)

Password Policy

All passwords must meet the following requirements enforced by the custom `@Password` validator:

Requirement	Rule
Minimum Length	8 characters
Uppercase Letters	At least one (A-Z)
Lowercase Letters	At least one (a-z)
Digits	At least one (0-9)
Special Characters	At least one from: <code>!@#\$%^&*()-_+=[]{};:'" .,<>/?~</code>